

1. JS Basics (1-5)

```
// 1. Hello World
console.log("Hello World");

// 2. Factorial
function factorial(n) {
  if (n === 0 || n === 1) return 1;
  return n * factorial(n - 1);
}

// 3. Reverse String
function reverseString(str) {
  return str.split('').reverse().join('');
}

// 4. Prime Number
function isPrime(n) {
  if (n <= 1) return false;
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) return false;
  }
  return true;
}

// 5. GCD / HCF
function getGCD(a, b) {
  if (b === 0) return a;
  return getGCD(b, a % b);
}
```

2. Funcs & Modules (6-7)

```
// --- mathModule.js ---
module.exports = {
  add: function(a, b) {
    return a + b;
  }
};

// --- app.js ---
const math = require('./mathModule');

// Normal Function
function normalFunc(msg) {
  return "Normal: " + msg;
}

// Arrow Function
const arrowFunc = (msg) => {
  return "Arrow: " + msg;
};

console.log(math.add(10, 5));
```

3. File Operations (8)

```
const fs = require('fs');

// Write data to a file synchronously
fs.writeFileSync('test.txt', 'Hello FSD');
console.log("File saved!");

// Read data from the file
const data = fs.readFileSync('test.txt', 'utf8');
console.log("File content: ", data);
```

4. HTTP Server & Query (9-12)

```
const http = require('http');
const url = require('url');

// Create Web Server
http.createServer(function(req, res) {
  // Parse the URL and query string
  const parsed = url.parse(req.url, true);

  res.writeHead(200, {'Content-Type': 'text/html'});

  // Routing logic
  if (parsed.pathname === '/login') {
    res.end('<b>Login Form Page</b>');
  }
  else if (parsed.pathname === '/query') {
    // Read ?name=chethan from URL
    let name = parsed.query.name;
    res.end('Hello ' + name);
  }
  else {
    res.end('Basic Web Server');
  }
}).listen(3000);
```

5. Events & Callbacks (15)

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Define the callback for the event
emitter.on('greet', function(name) {
  console.log("Event triggered! Hi " + name);
});

// Fire the event
emitter.emit('greet', 'Chethan');
```

6. MongoDB CRUD (13,14,16)

```
// These are MongoDB Shell Commands

// 1. Insert Data (Create)
db.students.insertOne({name: "Chethan", marks: 90})

// 2. Read Data
db.students.find({name: "Chethan"})

// 3. Update Data
db.students.updateOne(
  {name: "Chethan"},
  {$set: {marks: 95}}
)

// 4. Delete Data
db.students.deleteOne({name: "Chethan"})

// 5. Count, Sort, Skip, Limit
db.students.countDocuments()

db.students.find()
  .sort({marks: -1}) // Sort descending
  .skip(2)           // Skip first 2
  .limit(5)          // Show only 5
```

7. Express Routing (17)

```
const express = require('express');
const app = express();

// Basic GET Route
app.get('/', function(req, res) {
  res.send('Hello Express World!');
});

// Route with a Parameter (e.g., /user/123)
app.get('/user/:id', function(req, res) {
  let userId = req.params.id;
  res.send('You requested User ID: ' + userId);
});

// Start the server
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

8. AngularJS (18-19)

```
<!-- HTML Setup -->
<div ng-app="myApp" ng-controller="myCtrl">

  <!-- One-way Binding -->
  <p>Name: {{ username }}</p>

  <!-- Two-way Binding -->
  <input ng-model="username" placeholder="Type here">

</div>

<!-- Script Setup -->
<script src="angular.min.js"></script>
<script>
  // Define App and Controller
  const app = angular.module('myApp', []);

  app.controller('myCtrl', function($scope) {
    // Initial variable value
    $scope.username = "Student";
  });
</script>
```

9. ReactJS (Props, Life, Form)

```
import React, { Component, useState } from 'react';

// 1. Functional Component with Props
function WelcomeMsg(props) {
  return <h1>Hello, {props.name}</h1>;
}

// 2. Class Component with Lifecycle
class LifecycleDemo extends Component {
  componentDidMount() {
    // Runs after component loads
    console.log("Component has mounted!");
  }
  render() {
    return <p>I am a Class Component</p>;
  }
}

// 3. Form Validation with Hooks
function SimpleForm() {
  // State for input value
  const [val, setVal] = useState("");

  function handleSubmit(event) {
    event.preventDefault(); // Stop reload

    // Validation
    if (val === "") {
      alert("Error: Input cannot be empty!");
    } else {
      alert("Form Submitted: " + val);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        onChange={(e) => setVal(e.target.value)}
      />
      <button type="submit">Submit</button>
    </form>
  );
}
```